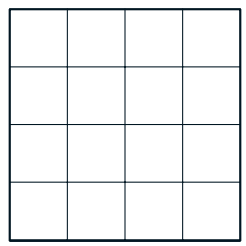
sum making up the approximate value function (9.8) is almost trivial to compute. Rather than performing n multiplications and additions, one simply computes the indices of the $m \ll n$ present features and then adds up the m corresponding components of the parameter vector. The eligibility trace computation (9.7) is also simplified because the components of the gradient, $\nabla\hat{v}(s,\mathbf{w})$, are also usually 0, and otherwise 1.

The computation of the indices of the present features is particularly easy if gridlike tilings are used. The ideas and techniques here are best illustrated by examples. Suppose we address a task with two continuous state variables. Then the simplest way to tile the space is with a uniform two-dimensional grid:

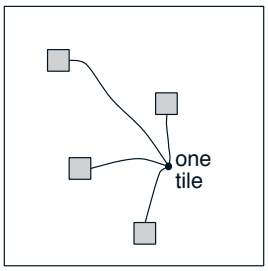


Given the x and y coordinates of a point in the space, it is computationally easy to determine the index of the tile it is in. When multiple tilings are used, each is offset by a different amount, so that each cuts the space in a different way. In the example shown in Figure 9.5, an extra row and an extra column of tiles have been added to the grid so that no points are left uncovered. The two tiles highlighted are those that are present in the state indicated by the X. The different tilings may be offset by random amounts, or by cleverly designed deterministic strategies (simply offsetting each dimension by the same increment is known not to be a good idea). The effects on generalization and asymptotic accuracy illustrated in Figures 9.3 and 9.4 apply here as well. The width and shape of the tiles should be chosen to match the width of generalization that one expects to be appropriate. The number of tilings should be chosen to influence the density of tiles. The denser the tiling, the finer and more accurately the desired function can be approximated, but the greater the computational costs.

It is important to note that the tilings can be arbitrary and need not be uniform grids. Not only can the tiles be strangely shaped, as in Figure 9.6a, but they can be shaped and distributed to give particular kinds of generalization. For example, the stripe tiling in Figure 9.6b will promote generalization along the vertical dimension and discrimination along the horizontal dimension, particularly on the left. The diagonal stripe tiling in Figure 9.6c will promote generalization along one diagonal. In higher dimensions, axis-aligned stripes correspond to ignoring some of the dimensions in some of the tilings,

that is, to hyperplanar slices.

Another important trick for reducing memory requirements is *hashing*—a consistent pseudo-random collapsing of a large tiling into a much smaller set of tiles. Hashing produces tiles consisting of noncontiguous, disjoint regions randomly spread throughout the state space, but that still form an exhaustive tiling. For example, one tile might consist of the four subtiles shown below:



Through hashing, memory requirements are often reduced by large factors with little loss of performance. This is possible because high resolution is needed in only a small fraction of the state space. Hashing frees us from the curse of dimensionality in the sense that memory requirements need not be exponential in the number of dimensions, but need merely match the real demands of the task. Good public-domain implementations of tile coding, including hashing, are widely available.

Radial Basis Functions

Radial basis functions (RBFs) are the natural generalization of coarse coding to continuous-valued features. Rather than each feature being either 0 or 1, it can be anything in the interval $[0, 1]$, reflecting various *degrees* to which the feature is present. A typical RBF feature, i, has a Gaussian (bell-shaped) response $x_i(s)$ dependent only on the distance between the state, s, and the

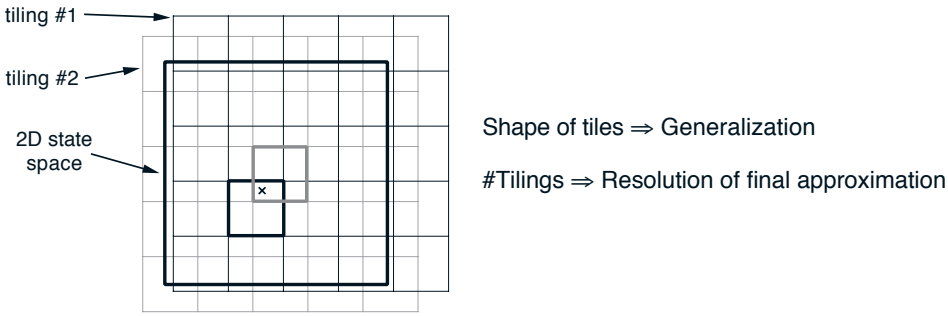


Figure 9.5: Multiple, overlapping gridtilings.

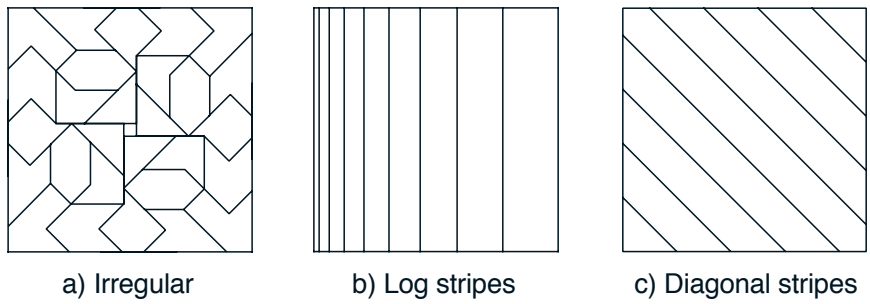


Figure 9.6: Tilings.

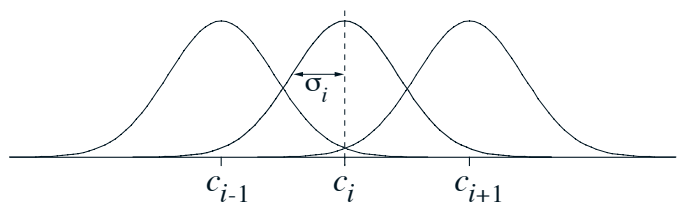


Figure 9.7: One-dimensional radial basis functions.

feature's prototypical or center state, c_i, and relative to the feature's width, σ_i:

$$x_i(s) = \exp\left(-\frac{||s - c_i||^2}{2\sigma_i^2}\right).$$

The norm or distance metric of course can be chosen in whatever way seems most appropriate to the states and task at hand. Figure 9.7 shows a one-dimensional example with a Euclidean distance metric.

An *RBF network* is a linear function approximator using RBFs for its features. Learning is defined by equations (9.3) and (9.8), exactly as in other linear function approximators. The primary advantage of RBFs over binary features is that they produce approximate functions that vary smoothly and are differentiable. In addition, some learning methods for RBF networks change the centers and widths of the features as well. Such nonlinear methods may be able to fit the target function much more precisely. The downside to RBF networks, and to nonlinear RBF networks especially, is greater computational complexity and, often, more manual tuning before learning is robust and efficient.

Kanerva Coding

On tasks with very high dimensionality, say *hundreds* of dimensions, tile coding and RBF networks become impractical. If we take either method at face value, its computational complexity increases exponentially with the number of dimensions. There are a number of tricks that can reduce this growth (such as hashing), but even these become impractical after a few tens of dimensions.

On the other hand, some of the general ideas underlying these methods can be practical for high-dimensional tasks. In particular, the idea of representing states by a list of the features present and then mapping those features linearly to an approximation may scale well to large tasks. The key is to keep the number of features from scaling explosively. Is there any reason to think this might be possible?

First we need to establish some realistic expectations. Roughly speaking, a function approximator of a given complexity can only accurately approximate target functions of comparable complexity. But as dimensionality increases, the size of the state space inherently increases exponentially. It is reasonable to assume that in the worst case the complexity of the target function scales like the size of the state space. Thus, if we focus the worst case, then there is no solution, no way to get good approximations for high-dimensional tasks without using resources exponential in the dimension.

A more useful way to think about the problem is to focus on the complexity of the target function as separate and distinct from the size and dimensionality of the state space. The size of the state space may give an upper bound on complexity, but short of that high bound, complexity and dimension can be unrelated. For example, one might have a 1000-dimensional task where only one of the dimensions happens to matter. Given a certain level of complexity, we then seek to be able to accurately approximate any target function of that complexity or less. As the target level of complexity increases, we would like to get by with a proportionate increase in computational resources.

From this point of view, the real source of the problem is the complexity of the target function, or of a reasonable approximation of it, not the dimensionality of the state space. Thus, adding dimensions, such as new sensors or new features, to a task should be almost without consequence if the complexity of the needed approximations remains the same. The new dimensions may even make things easier if the target function can be simply expressed in terms of them. Unfortunately, methods like tile coding and RBF coding do not work this way. Their complexity increases exponentially with dimensionality even if the complexity of the target function does not. For these methods, dimensionality itself is still a problem. We need methods whose complexity is unaffected

by dimensionality per se, methods that are limited only by, and scale well with, the complexity of what they approximate.

One simple approach that meets these criteria, which we call *Kanerva coding*, is to choose binary features that correspond to particular *prototype states*. For definiteness, let us say that the prototypes are randomly selected from the entire state space. The receptive field of such a feature is all states sufficiently close to the prototype. Kanerva coding uses a different kind of distance metric than in is used in tile coding and RBFs. For definiteness, consider a *binary* state space and the *hamming distance*, the number of bits at which two states differ. States are considered similar if they agree on enough dimensions, even if they are totally different on others.

The strength of Kanerva coding is that the complexity of the functions that can be learned depends entirely on the number of features, which bears no necessary relationship to the dimensionality of the task. The number of features can be more or less than the number of dimensions. Only in the worst case must it be exponential in the number of dimensions. Dimensionality itself is thus no longer a problem. Complex functions are still a problem, as they have to be. To handle more complex tasks, a Kanerva coding approach simply needs more features. There is not a great deal of experience with such systems, but what there is suggests that their abilities increase in proportion to their computational resources. This is an area of current research, and significant improvements in existing methods can still easily be found.

9.4 Control with Function Approximation

We now extend value prediction methods using function approximation to control methods, following the pattern of GPI. First we extend the state-value prediction methods to action-value prediction methods, then we combine them with policy improvement and action selection techniques. As usual, the problem of ensuring exploration is solved by pursuing either an on-policy or an off-policy approach.

The extension to action-value prediction is straightforward. In this case it is the approximate action-value function, $\hat{q} \approx q_\pi$, that is represented as a parameterized functional form with parameter vector $\mathbf{w}$. Whereas before we considered random training examples of the form $S_t \mapsto V_t$, now we consider examples of the form $S_t, A_t \mapsto Q_t$. The target output, Q_t, can be any approximation of $q_\pi(S_t, A_t)$, including the usual backed-up values such as the full Monte Carlo return, G_t, or the one-step Sarsa-style return, $G_{t+1} + \gamma\hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t)$.

The general gradient-descent update for action-value prediction is

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha\Big[Q_t - \hat{q}(S_t, A_t, \mathbf{w}_t)\Big]\nabla\hat{q}(S_t, A_t, \mathbf{w}_t).$$

For example, the backward view of the action-value method analogous to TD(λ) is

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha\delta_t\mathbf{e}_t,$$

where

$$\delta_t = R_{t+1} + \gamma\hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t),$$

and

$$\mathbf{e}_t = \gamma\lambda\mathbf{e}_{t-1} + \nabla\hat{q}(S_t, A_t, \mathbf{w}_t),$$

with $\mathbf{e}_0 = \mathbf{0}$. We call this method *gradient-descent Sarsa(λ)*, particularly when it is elaborated to form a full control method. For a constant policy, this method converges in the same way that TD(λ) does, with the same kind of error bound (9.9).

To form control methods, we need to couple such action-value prediction methods with techniques for policy improvement and action selection. Suitable techniques applicable to continuous actions, or to actions from large discrete sets, are a topic of ongoing research with as yet no clear resolution. On the other hand, if the action set is discrete and not too large, then we can use the techniques already developed in previous chapters. That is, for each possible action, a, available in the current state, S_t, we can compute $\hat{q}(S_t, a, \mathbf{w}_t)$ and then find the greedy action $a^*_t = \operatorname{argmax}_a \hat{q}(S_t, a, \mathbf{w}_t)$. Policy improvement is done by changing the estimation policy to the greedy policy (in off-policy methods) or to a soft approximation of the greedy policy such as the ε-greedy policy (in on-policy methods). Actions are selected according to this same policy in on-policy methods, or by an arbitrary policy in off-policy methods.

Figures 9.8 and 9.9 show examples of on-policy (Sarsa(λ)) and off-policy (Watkins's Q(λ)) control methods using function approximation. Both methods use linear, gradient-descent function approximation with binary features, such as in tile coding and Kanerva coding. Both methods use an ε-greedy policy for action selection, and the Sarsa method uses it for GPI as well. Both compute the sets of present features, $\mathcal{F}_a$, corresponding to the current state and all possible actions, a. If the value function for each action is a separate linear function of the same features (a common case), then the indices of the $\mathcal{F}_a$ for each action are essentially the same, simplifying the computation significantly.

```
Let w and e be vectors with one component for each possible feature
Let F_a, for every possible action a, be a set of feature indices, initially empty
Initialize w as appropriate for the problem, e.g., w = 0
Repeat (for each episode):
    e = 0
    S, A ← initial state and action of episode        (e.g., ε-greedy)
    F_A ← set of features present in S, A
    Repeat (for each step of episode):
        For all i ∈ F_A:
            e_i ← e_i + 1                  (accumulating traces)
            or e_i ← 1                     (replacing traces)
        Take action A, observe reward, R, and next state, S'
        δ ← R − Σ_{i∈F_A} w_i
        If S' is terminal, then w ← w + αδe; go to next episode
        For all a ∈ A(S'):
            F_a ← set of features present in S', a
            Q_a ← Σ_{i∈F_a} w_i
        A' ← new action in S' (e.g., ε-greedy)
        δ ← δ + γQ_{A'}
        w ← w + αδe
        e ← γλe
        S ← S'
        A ← A'
```

Figure 9.8: Linear, gradient-descent Sarsa(λ) with binary features and ε-greedy policy. Updates for both accumulating and replacing traces are specified.

```
Let w and e be vectors with one component for each possible feature
Let F_a, for every possible action a, be a set of feature indices, initially empty
Initialize w as appropriate for the problem, e.g., w = 0
Repeat (for each episode):
    e = 0
    S ← initial state of episode
    Repeat (for each step of episode):
        For all a ∈ A(S):
            F_a ← set of features present in S, a
            Q_a ← Σ_{i∈F_a} w_i
        A* ← arg max_a Q_a
        A ← A* with prob. 1 − ε, else a random action ∈ A(S)
        If A ≠ A*, then e = 0
        Take action A, observe reward, R, and next state, S'
        δ ← R − Q_A
        For all i ∈ F_A:
            e_i ← e_i + 1                (accumulating traces)
            or e_i ← 1                   (replacing traces)
        If S' is terminal, then w ← w + αδe; go to next episode
        For all a ∈ A(S'):
            F_a ← set of features present in S', a
            Q_a ← Σ_{i∈F_a} w_i
        δ ← δ + γ max_{a∈A(S')} Q_a
        w ← w + αδe
        e ← γλe
        S ← S'
```

Figure 9.9: A linear, gradient-descent version of Watkins's Q(λ) with binary features and ε-greedy policy. Updates for both accumulating and replacing traces are specified.

All the methods we have discussed above have used *accumulating* eligibility traces. Although replacing traces (Section 7.8) are known to have advantages in tabular methods, replacing traces do not directly extend to the use of function approximation. Recall that the idea of replacing traces is to reset a state's trace to 1 each time it is visited instead of incrementing it by 1. But with function approximation there is no single trace corresponding to a state, just a trace for each component of $\mathbf{w}$, which corresponds to many states. One approach that seems to work well for linear, gradient-descent function approximation methods with binary features is to treat the features as if they were states for the purposes of replacing traces. That is, each time a state is encountered that has feature i, the trace for feature i is set to 1 rather than being incremented by 1, as it would be with accumulating traces.

When working with state–action traces, it may also be useful to clear (set to zero) the traces of all nonselected actions in the states encountered (see Section 7.8). This idea can also be extended to the case of linear function approximation with binary features. For each state encountered, we first clear the traces of all features for the state and the actions not selected, then we set to 1 the traces of the features for the state and the action that was selected. As we noted for the tabular case, this may or may not be the best way to proceed when using replacing traces. A procedural specification of both kinds of traces, including the optional clearing for nonselected actions, is given for the Sarsa algorithm in Figure 9.8.

Example 9.2: Mountain–Car Task Consider the task of driving an underpowered car up a steep mountain road, as suggested by the diagram in the upper left of Figure 9.10. The difficulty is that gravity is stronger than the car's engine, and even at full throttle the car cannot accelerate up the steep slope. The only solution is to first move away from the goal and up the opposite slope on the left. Then, by applying full throttle the car can build up enough inertia to carry it up the steep slope even though it is slowing down the whole way. This is a simple example of a continuous control task where things have to get worse in a sense (farther from the goal) before they can get better. Many control methodologies have great difficulties with tasks of this kind unless explicitly aided by a human designer.

The reward in this problem is -1 on all time steps until the car moves past its goal position at the top of the mountain, which ends the episode. There are three possible actions: full throttle forward ($+1$), full throttle reverse (-1), and zero throttle (0). The car moves according to a simplified physics. Its position, p_t, and velocity, $\dot{p}_t$, are updated by

$$p_{t+1} = \mathit{bound}\big[p_t + \dot{p}_{t+1}\big]$$

$$\dot{p}_{t+1} = \mathit{bound}\big[\dot{p}_t + 0.001A_t - 0.0025\cos(3p_t)\big],$$

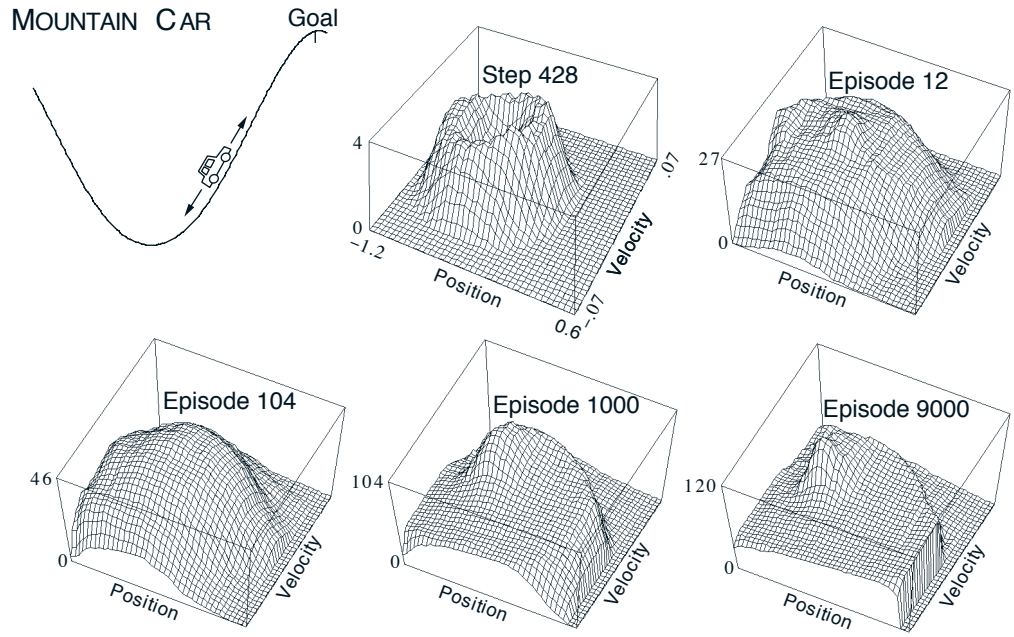


Figure 9.10: The mountain–car task (upper left panel) and the cost-to-go function ($-\max_a \hat{q}(s,a,\mathbf{w})$) learned during one run.

where the *bound* operation enforces $-1.2 \leq p_{t+1} \leq 0.5$ and $-0.07 \leq \dot{p}_{t+1} \leq 0.07$. When p_{t+1} reached the left bound, $\dot{p}_{t+1}$ was reset to zero. When it reached the right bound, the goal was reached and the episode was terminated. Each episode started from a random position and velocity uniformly chosen from these ranges. To convert the two continuous state variables to binary features, we used gridtilings as in Figure 9.5. We used ten 9×9 tilings, each offset by a random fraction of a tile width.

The Sarsa algorithm in Figure 9.8 (using replace traces and the optional clearing) readily solved this task, learning a near optimal policy within 100 episodes. Figure 9.10 shows the negative of the value function (the *cost-to-go* function) learned on one run, using the parameters $\lambda = 0.9$, $\varepsilon = 0$, and $\alpha = 0.05$ (e.g., $\frac{0.5}{m}$). The initial action values were all zero, which was optimistic (all true values are negative in this task), causing extensive exploration to occur even though the exploration parameter, ε, was 0. This can be seen in the middle-top panel of the figure, labeled "Step 428." At this time not even one episode had been completed, but the car has oscillated back and forth in the valley, following circular trajectories in state space. All the states visited frequently are valued worse than unexplored states, because the actual rewards have been worse than what was (unrealistically) expected. This continually drives the agent away from wherever it has been, to explore new states, until a solution is found. Figure 9.11 shows the results of a detailed study of the

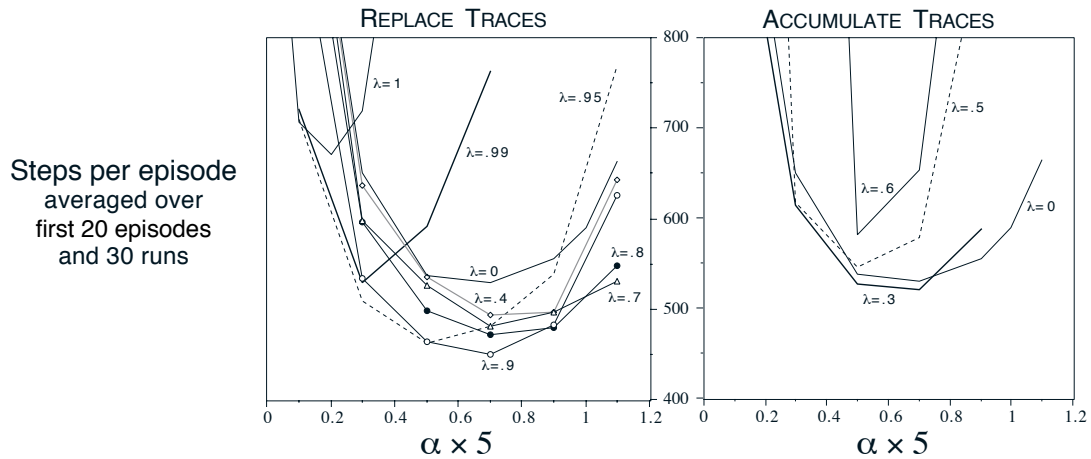


Figure 9.11: The effect of α, λ, and the kind of traces on early performance on the mountain–car task. This study used five 9×9 tilings.

effect of the parameters α and λ, and of the kind of traces, on the rate of learning on this task. ■

9.5 Should We Bootstrap?

At this point you may be wondering why we bother with bootstrapping methods at all. Nonbootstrapping methods can be used with function approximation more reliably and over a broader range of conditions than bootstrapping methods. Nonbootstrapping methods achieve a lower asymptotic error than bootstrapping methods, even when backups are done according to the on-policy distribution. By using eligibility traces and $\lambda = 1$, it is even possible to implement nonbootstrapping methods on-line, in a step-by-step incremental manner. Despite all this, in practice bootstrapping methods are usually the methods of choice.

In empirical comparisons, bootstrapping methods usually perform much better than nonbootstrapping methods. A convenient way to make such comparisons is to use a TD method with eligibility traces and vary λ from 0 (pure bootstrapping) to 1 (pure nonbootstrapping). Figure 9.12 summarizes a collection of such results. In all cases, performance became much worse as λ approached 1, the nonbootstrapping case. The example in the upper right of the figure is particularly significant in this regard. This is a policy evaluation (prediction) task and the performance measure used is RMSE (at the end of each episode, averaged over the first 20 episodes). Asymptotically, the $\lambda = 1$ case must be best according to this measure, but here, short of the asymptote,

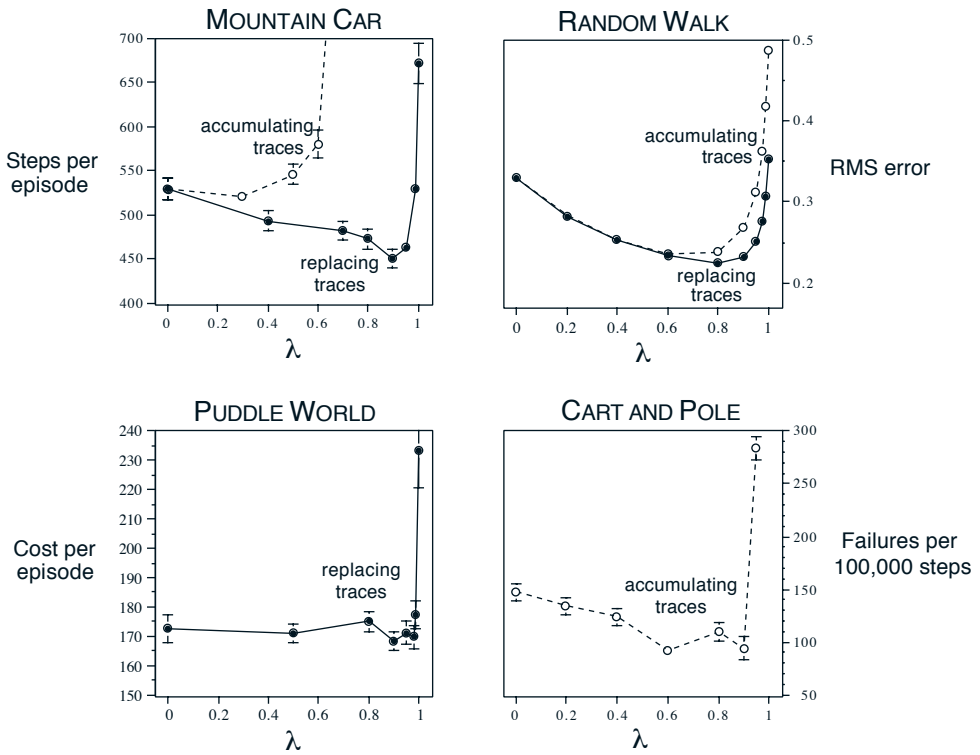


Figure 9.12: The effect of λ on reinforcement learning performance. In all cases, the better the performance, the *lower* the curve. The two left panels are applications to simple continuous-state control tasks using the Sarsa(λ) algorithm and tile coding, with either replacing or accumulating traces (Sutton, 1996). The upper-right panel is for policy evaluation on a random walk task using TD(λ) (Singh and Sutton, 1996). The lower right panel is unpublished data for the pole-balancing task (Example 3.4) from an earlier study (Sutton, 1984).

we see it performing much worse.

At this time it is unclear why methods that involve some bootstrapping perform so much better than pure nonbootstrapping methods. It could be that bootstrapping methods learn faster, or it could be that they actually learn something better than nonbootstrapping methods. The available results indicate that nonbootstrapping methods are better than bootstrapping methods at reducing RMSE from the true value function, but reducing RMSE is not necessarily the most important goal. For example, if you add 1000 to the true action-value function at all state–action pairs, then it will have very poor RMSE, but you will still get the optimal policy. Nothing quite that simple is going on with bootstrapping methods, but they do seem to do something right. We expect the understanding of these issues to improve as research continues.

9.6 Summary

Reinforcement learning systems must be capable of *generalization* if they are to be applicable to artificial intelligence or to large engineering applications. To achieve this, any of a broad range of existing methods for *supervised-learning function approximation* can be used simply by treating each backup as a training example. *Gradient-descent methods*, in particular, allow a natural extension to function approximation of all the techniques developed in previous chapters, including eligibility traces. *Linear* gradient-descent methods are particularly appealing theoretically and work well in practice when provided with appropriate features. Choosing the features is one of the most important ways of adding prior domain knowledge to reinforcement learning systems. Linear methods include radial basis functions, tile coding, and Kanerva coding. Backpropagation methods for multilayer neural networks are methods for *nonlinear* gradient-descent function approximation.

For the most part, the extension of reinforcement learning prediction and control methods to gradient-descent forms is straightforward for the on-policy case. On-policy bootstrapping methods converge reliably with linear gradient-descent function approximation to a solution with mean-squared error bounded by $\frac{1-\gamma\lambda}{1-\gamma}$ times the minimum possible error. Bootstrapping methods are of persistent interest in reinforcement learning, despite their limited theoretical guarantees, because in practice they usually work significantly better than nonbootstrapping methods. The off-policy case involves considerably greater subtlety and is postponed to a later (future) chapter.

Bibliographical and Historical Remarks

Despite our treatment of generalization and function approximation late in the book, they have always been an integral part of reinforcement learning. It is only in the last decade or less that the field has focused on the tabular case, as we have here for the first seven chapters. Bertsekas and Tsitsiklis (1996) present the state of the art in function approximation in reinforcement learning, and the collection of papers by Boyan, Moore, and Sutton (1995) is also useful. Some of the early work with function approximation in reinforcement learning is discussed at the end of this section.

9.2 Gradient-descent methods for the minimizing mean-squared error in supervised learning are well known. Widrow and Hoff (1960) introduced the least-mean-square (LMS) algorithm, which is the prototypical incremental gradient-descent algorithm. Details of this and related algorithms are provided in many texts (e.g., Widrow and Stearns, 1985; Bishop, 1995; Duda and Hart, 1973).

Gradient-descent analyses of TD learning date back at least to Sutton (1988). Methods more sophisticated than the simple gradient-descent methods covered in this section have also been studied in the context of reinforcement learning, such as quasi-Newton methods (Werbos, 1990) and recursive-least-squares methods (Bradtke, 1993, 1994; Bradtke and Barto, 1996; Bradtke, Ydstie, and Barto, 1994). Bertsekas and Tsitsiklis (1996) provide a good discussion of these methods.

The earliest use of state aggregation in reinforcement learning may have been Michie and Chambers's BOXES system (1968). The theory of state aggregation in reinforcement learning has been developed by Singh, Jaakkola, and Jordan (1995) and Tsitsiklis and Van Roy (1996).

9.3 TD(λ) with linear gradient-descent function approximation was first explored by Sutton (1984, 1988), who proved convergence of TD(0) in the mean to the minimal RMSE solution for the case in which the feature vectors, $\{\mathbf{x}(s) : s \in \mathcal{S}\}$, are linearly independent. Convergence with probability 1 for general λ was proved by several researchers at about the same time (Peng, 1993; Dayan and Sejnowski, 1994; Tsitsiklis, 1994; Gurvits, Lin, and Hanson, 1994). In addition, Jaakkola, Jordan, and Singh (1994) proved convergence under on-line updating. All of these results assumed linearly independent feature vectors, which implies at least as many components to $\mathbf{w}_t$ as there are states. Convergence of linear TD(λ) for the more interesting case of general (dependent) feature vectors was first shown by Dayan (1992). A significant generalization

and strengthening of Dayan's result was proved by Tsitsiklis and Van Roy (1997). They proved the main result presented in Section 9.2, the bound on the asymptotic error of TD(λ) and other bootstrapping methods. Recently they extended their analysis to the undiscounted continuing case (Tsitsiklis and Van Roy, 1999).

Our presentation of the range of possibilities for linear function approximation is based on that by Barto (1990). The term *coarse coding* is due to Hinton (1984), and our Figure 9.2 is based on one of his figures. Waltz and Fu (1965) provide an early example of this type of function approximation in a reinforcement learning system.

Tile coding, including hashing, was introduced by Albus (1971, 1981). He described it in terms of his "cerebellar model articulator controller," or CMAC, as tile coding is known in the literature. The term "tile coding" is new to this book, though the idea of describing CMAC in these terms is taken from Watkins (1989). Tile coding has been used in many reinforcement learning systems (e.g., Shewchuk and Dean, 1990; Lin and Kim, 1991; Miller, Scalera, and Kim, 1994; Sofge and White, 1992; Tham, 1994; Sutton, 1996; Watkins, 1989) as well as in other types of learning control systems (e.g., Kraft and Campagna, 1990; Kraft, Miller, and Dietz, 1992).

Function approximation using radial basis functions (RBFs) has received wide attention ever since being related to neural networks by Broomhead and Lowe (1988). Powell (1987) reviewed earlier uses of RBFs, and Poggio and Girosi (1989, 1990) extensively developed and applied this approach.

What we call "Kanerva coding" was introduced by Kanerva (1988) as part of his more general idea of *sparse distributed memory*. A good review of this and related memory models is provided by Kanerva (1993). This approach has been pursued by Gallant (1993) and by Sutton and Whitehead (1993), among others.

9.4 Q(λ) with function approximation was first explored by Watkins (1989). Sarsa(λ) with function approximation was first explored by Rummery and Niranjan (1994). The mountain–car example is based on a similar task studied by Moore (1990). The results on it presented here are from Sutton (1996) and Singh and Sutton (1996).

Convergence of the Sarsa control method presented in this section has not been proved. The Q-learning control method is now known not to be sound and will diverge for some problems. Convergence results for control methods with state aggregation and other special kinds of

function approximation are proved by Tsitsiklis and Van Roy (1996), Singh, Jaakkola, and Jordan (1995), and Gordon (1995).

The use of function approximation in reinforcement learning goes back to the early neural networks of Farley and Clark (1954; Clark and Farley, 1955), who used reinforcement learning to adjust the parameters of linear threshold functions representing policies. The earliest example we know of in which function approximation methods were used for learning value functions was Samuel's checkers player (1959, 1967). Samuel followed Shannon's (1950) suggestion that a value function did not have to be exact to be a useful guide to selecting moves in a game and that it might be approximated by linear combination of features. In addition to linear function approximation, Samuel experimented with lookup tables and hierarchical lookup tables called signature tables (Griffith, 1966, 1974; Page, 1977; Biermann, Fairfield, and Beres, 1982).

At about the same time as Samuel's work, Bellman and Dreyfus (1959) proposed using function approximation methods with DP. (It is tempting to think that Bellman and Samuel had some influence on one another, but we know of no reference to the other in the work of either.) There is now a fairly extensive literature on function approximation methods and DP, such as multigrid methods and methods using splines and orthogonal polynomials (e.g., Bellman and Dreyfus, 1959; Bellman, Kalaba, and Kotkin, 1973; Daniel, 1976; Whitt, 1978; Reetz, 1977; Schweitzer and Seidmann, 1985; Chow and Tsitsiklis, 1991; Kushner and Dupuis, 1992; Rust, 1996).

Holland's (1986) classifier system used a selective feature-match technique to generalize evaluation information across state–action pairs. Each classifier matched a subset of states having specified values for a subset of features, with the remaining features having arbitrary values ("wild cards"). These subsets were then used in a conventional state-aggregation approach to function approximation. Holland's idea was to use a genetic algorithm to evolve a set of classifiers that collectively would implement a useful action-value function. Holland's ideas influenced the early research of the authors on reinforcement learning, but we focused on different approaches to function approximation. As function approximators, classifiers are limited in several ways. First, they are state-aggregation methods, with concomitant limitations in scaling and in representing smooth functions efficiently. In addition, the matching rules of classifiers can implement only aggregation boundaries that are parallel to the feature axes. Perhaps the most important limitation of conventional classifier systems is that the classifiers are learned via the genetic algorithm, an evolutionary method. As we discussed in Chapter 1, there is available during learning much more detailed information about how to learn than can be

used by evolutionary methods. This perspective led us to instead adapt supervised learning methods for use in reinforcement learning, specifically gradient-descent and neural network methods. These differences between Holland's approach and ours are not surprising because Holland's ideas were developed during a period when neural networks were generally regarded as being too weak in computational power to be useful, whereas our work was at the beginning of the period that saw widespread questioning of that conventional wisdom. There remain many opportunities for combining aspects of these different approaches.

A number of reinforcement learning studies using function approximation methods that we have not covered previously should be mentioned. Barto, Sutton, and Brouwer (1981) and Barto and Sutton (1981b) extended the idea of an associative memory network (e.g., Kohonen, 1977; Anderson, Silverstein, Ritz, and Jones, 1977) to reinforcement learning. Hampson (1983, 1989) was an early proponent of multilayer neural networks for learning value functions. Anderson (1986, 1987) coupled a TD algorithm with the error backpropagation algorithm to learn a value function. Barto and Anandan (1985) introduced a stochastic version of Widrow, Gupta, and Maitra's (1973) *selective bootstrap algorithm*, which they called the *associative reward-penalty* (A_{R-P}) *algorithm*. Williams (1986, 1987, 1988, 1992) extended this type of algorithm to a general class of REINFORCE algorithms, showing that they perform stochastic gradient ascent on the expected reinforcement. Gullapalli (1990) and Williams devised algorithms for learning generalizing policies for the case of continuous actions. Phansalkar and Thathachar (1995) proved both local and global convergence theorems for modified versions of REINFORCE algorithms. Christensen and Korf (1986) experimented with regression methods for modifying coefficients of linear value function approximations in the game of chess. Chapman and Kaelbling (1991) and Tan (1991) adapted decision-tree methods for learning value functions. Explanation-based learning methods have also been adapted for learning value functions, yielding compact representations (Yee, Saxena, Utgoff, and Barto, 1990; Dietterich and Flann, 1995).

Exercises

Exercise 9.1 Show that table-lookup TD(λ) is a special case of general TD(λ) as given by equations (9.5–9.7).

Exercise 9.2 *State aggregation* is a simple form of generalizing function approximation in which states are grouped together, with one table entry (value estimate) used for each group. Whenever a state in a group is encountered, the group's entry is used to determine the state's value, and when the state is up-

dated, the group's entry is updated. Show that this kind of state aggregation is a special case of a gradient method such as (9.4).

Exercise 9.3 The equations given in this section are for the on-line version of gradient-descent TD(λ). What are the equations for the *off-line* version? Give a complete description specifying the new weight vector at the end of an episode, $\mathbf{w}'$, in terms of the weight vector used during the episode, $\mathbf{w}$. Start by modifying a forward-view equation for TD(λ), such as (9.4).

Exercise 9.4 For off-line updating, show that equations (9.5–9.7) produce updates identical to (9.4).

Exercise 9.5 How could we reproduce the tabular case within the linear framework?

Exercise 9.6 How could we reproduce the state aggregation case (see Exercise 8.4) within the linear framework?

Exercise 9.7 Suppose we believe that one of two state dimensions is more likely to have an effect on the value function than is the other, that generalization should be primarily across this dimension rather than along it. What kind of tilings could be used to take advantage of this prior knowledge?

Chapter 10

Off-policy Approximation of Action Values

Chapter 11

Policy Approximation

All of the methods we have considered so far in this book have learned the values of states or state–action pairs. To use them for control, we learned the values of state–action pairs, and then used those action values directly to implement the policy (e.g., ε-greedy) and select actions. All methods of this form can be called *action-value methods*.

In this chapter we explore methods that are not action-value methods. They may still compute action (or state) values, but they do not use them directly to select actions. Instead, the policy is represented directly, with its own weights independent of any value function.

11.1 Actor–Critic Methods

Actor–critic methods are TD methods that have a separate memory structure to explicitly represent the policy independent of the value function. The policy structure is known as the *actor*, because it is used to select actions, and the estimated value function is known as the *critic*, because it criticizes the actions made by the actor. Learning is always on-policy: the critic must learn about and critique whatever policy is currently being followed by the actor. The critique takes the form of a TD error. This scalar signal is the sole output of the critic and drives all learning in both actor and critic, as suggested by Figure 11.1.

Actor–critic methods are the natural extension of the idea of gradient-bandit methods (Section 2.7) to TD learning and to the full reinforcement learning problem. Typically, the critic is a state-value function. After each action selection, the critic evaluates the new state to determine whether things have gone better or worse than expected. That evaluation is the TD error:

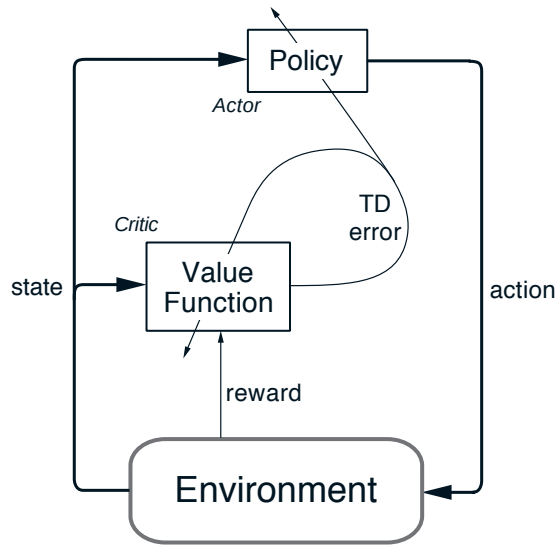


Figure 11.1: The actor–critic architecture.

$$\delta_t = R_{t+1} + \gamma V_t(S_{t+1}) - V(S_t),$$

where V_t is the value function implemented by the critic at time t. This TD error can be used to evaluate the action just selected, the action A_t taken in state S_t. If the TD error is positive, it suggests that the tendency to select A_t should be strengthened for the future, whereas if the TD error is negative, it suggests the tendency should be weakened. Suppose actions are generated by the Gibbs softmax method:

$$\pi_t(a|s) = \Pr\{A_t{=}a \mid S_t{=}s\} = \frac{e^{H_t(s,a)}}{\sum_b e^{H_t(s,b)}},$$

where the $H_t(s,a)$ are the values at time t of the modifiable policy parameters of the actor, indicating the tendency to select (*preference* for) each action a when in each state s at time t. Then the strengthening or weakening described above can be implemented by increasing or decreasing $H_t(S_t, A_t)$, for instance, by

$$H_{t+1}(S_t, A_t) = H_t(S_t, A_t) + \beta\delta_t,$$

where β is another positive step-size parameter.

This is just one example of an actor–critic method. Other variations select the actions in different ways, or use eligibility traces like those described in the

next chapter. Another common dimension of variation, as in reinforcement comparison methods, is to include additional factors varying the amount of credit assigned to the action taken, A_t. For example, one of the most common such factors is inversely related to the probability of selecting A_t, resulting in the update rule:

$$H_t(S_t, A_t) = H_t(S_t, A_t) + \beta\delta_t\Big[1 - \pi_t(A_t|S_t)\Big].$$

These issues were explored early on, primarily for the immediate reward case (Sutton, 1984; Williams, 1992) and have not been brought fully up to date.

Many of the earliest reinforcement learning systems that used TD methods were actor–critic methods (Witten, 1977; Barto, Sutton, and Anderson, 1983). Since then, more attention has been devoted to methods that learn action-value functions and determine a policy exclusively from the estimated values (such as Sarsa and Q-learning). This divergence may be just historical accident. For example, one could imagine intermediate architectures in which both an action-value function and an independent policy would be learned. In any event, actor–critic methods are likely to remain of current interest because of two significant apparent advantages:

- They require minimal computation in order to select actions. Consider a case where there are an infinite number of possible actions—for example, a continuous-valued action. Any method learning just action values must search through this infinite set in order to pick an action. If the policy is explicitly stored, then this extensive computation may not be needed for each action selection.
- They can learn an explicitly stochastic policy; that is, they can learn the optimal probabilities of selecting various actions. This ability turns out to be useful in competitive and non-Markov cases (e.g., see Singh, Jaakkola, and Jordan, 1994).

In addition, the separate actor in actor–critic methods makes them more appealing in some respects as psychological and biological models. In some cases it may also make it easier to impose domain-specific constraints on the set of allowed policies.

11.2 Eligibility Traces for Actor–Critic Methods

In this section we describe how to extend the actor–critic methods introduced in Section 11.1 to use eligibility traces. This is fairly straightforward. The

critic part of an actor–critic method is simply on-policy learning of v_π. The TD(λ) algorithm can be used for that, with one eligibility trace for each state. The actor part needs to use an eligibility trace for each state–action pair. Thus, an actor–critic method needs two sets of traces, one for each state and one for each state–action pair.

Recall that the one-step actor–critic method updates the actor by

$$H_{t+1}(s,a) = \begin{cases} H_t(s,a) + \alpha\,\delta_t & \text{if } a = A_t \text{ and } s = S_t \\ H_t(s,a) & \text{otherwise,} \end{cases}$$

where δ_t is the TD(λ) error (7.10), and $H_t(s,a)$ is the preference for taking action a at time t if in state s. The preferences determine the policy via, for example, a softmax method (Section 2.3). We generalize the above equation to use eligibility traces as follows:

$$H_{t+1}(s,a) = H_t(s,a) + \alpha\,\delta_t\,E_t(s,a), \tag{11.1}$$

where $E_t(s,a)$ denotes the trace at time t for state–action pair s,a. For the simplest case mentioned above, the trace can be updated as in Sarsa(λ).

In Section 11.1 we also discussed a more sophisticated actor–critic method that uses the update

$$H_{t+1}(s,a) = \begin{cases} H_t(s,a) + \alpha\,\delta_t[1 - \pi_t(a|s)] & \text{if } a = A_t \text{ and } s = S_t \\ H_t(s,a) & \text{otherwise.} \end{cases}$$

To generalize this equation to eligibility traces we can use the same update (11.1) with a slightly different trace. Rather than incrementing the trace by 1 each time a state–action pair occurs, it is updated by $1 - \pi_t(S_t, A_t)$:

$$E_t(s,a) = \begin{cases} \gamma\lambda E_{t-1}(s,a) + 1 - \pi_t(S_t, A_t) & \text{if } s = S_t \text{ and } a = A_t; \\ \gamma\lambda E_{t-1}(s,a) & \text{otherwise,} \end{cases} \tag{11.2}$$

for all s,a.

11.3 R-Learning and the Average-Reward Setting

When the policy is approximated, we generally have to abandon the discounted-reward setting that we have relied on up to now. We replace it with the *average-reward setting*, which we discuss in this section.

R-learning is an off-policy control method for the advanced version of the reinforcement learning problem in which one neither discounts nor divides

experience into distinct episodes with finite returns. In this *average-reward setting*, one seeks to maximize the average reward per time step. The value functions for a policy, π, are defined relative to the average expected reward per step under the policy, $\bar{r}(\pi)$:

$$\bar{r}(\pi) = \lim_{n\to\infty} \frac{1}{n} \sum_{t=1}^{n} \mathbb{E}_\pi[R_t] \,.$$

This average reward is well defined if we assume that the process is ergodic (nonzero probability of reaching any state from any other under any policy), and thus that $\bar{r}(\pi)$ does not depend on the starting state. From any state, in the long run the average reward is the same, but there is a transient. From some states better-than-average rewards are received for a while, and from others worse-than-average rewards are received. It is this transient that defines the value of a state:

$$v_\pi(s) = \sum_{k=1}^{\infty} \mathbb{E}_\pi[R_{t+k} - \bar{r}(\pi) \mid S_t = s] \,,$$

and the value of a state–action pair is similarly the transient difference in reward when starting in that state and taking that action:

$$q_\pi(s,a) = \sum_{k=1}^{\infty} \mathbb{E}_\pi[R_{t+k} - \bar{r}(\pi) \mid S_t = s, A_t = a] \,.$$

We call these *relative values* because they are relative to the average reward under the current policy.

There are subtle distinctions that need to be drawn between different kinds of optimality in the undiscounted continuing case. Nevertheless, for most practical purposes it may be adequate simply to order policies according to their average reward per time step, in other words, according to their $\bar{r}(\pi)$. For now let us consider all policies that attain the maximal value of $\bar{r}(\pi)$ to be optimal.

Other than its use of relative values, R-learning is a standard TD control method based on off-policy GPI, much like Q-learning. It maintains two policies, a behavior policy and an estimation policy, plus an action-value function and an estimated average reward. The behavior policy is used to generate experience; it might be the ε-greedy policy with respect to the action-value function. The estimation policy is the one involved in GPI. It is typically the greedy policy with respect to the action-value function. If π is the estimation policy, then the action-value function, Q, is an approximation of q_π and the average reward, $\bar{R}$, is an approximation of $\bar{r}(\pi)$. The complete algorithm is given in Figure 11.2.

```
Initialize R̄ and Q(s,a), for all s,a, arbitrarily
Repeat forever:
    S ← current state
    Choose action A in S using behavior policy (e.g., ε-greedy)
    Take action A, observe R, S'
    δ ← R − R̄ + max_a Q(S',a) − Q(S,A)
    Q(S,A) ← Q(S,A) + αδ
    If Q(S,A) = max_a Q(S,a), then:
        R̄ ← R̄ + βδ
```

Figure 11.2: R-learning: An off-policy TD control algorithm for undiscounted, continuing tasks. The scalars α and β are step-size parameters.

Example 11.1: An Access-Control Queuing Task This is a decision task involving access control to a set of n servers. Customers of four different priorities arrive at a single queue. If given access to a server, the customers pay a reward of 1, 2, 4, or 8, depending on their priority, with higher priority customers paying more. In each time step, the customer at the head of the queue is either accepted (assigned to one of the servers) or rejected (removed from the queue). In either case, on the next time step the next customer in the queue is considered. The queue never empties, and the proportion of (randomly distributed) high priority customers in the queue is h. Of course a customer can be served only if there is a free server. Each busy server becomes free with probability p on each time step. Although we have just described them for definiteness, let us assume the statistics of arrivals and departures are unknown. The task is to decide on each step whether to accept or reject the next customer, on the basis of his priority and the number of free servers, so as to maximize long-term reward without discounting. Figure 11.3 shows the solution found by R-learning for this task with $n = 10$, $h = 0.5$, and $p = 0.06$. The R-learning parameters were $\alpha = 0.01$, $\beta = 0.01$, and $\epsilon = 0.1$. The initial action values and $\bar{R}$ were zero. ■

Exercises

***Exercise 11.1** Design an on-policy method for undiscounted, continuing tasks.

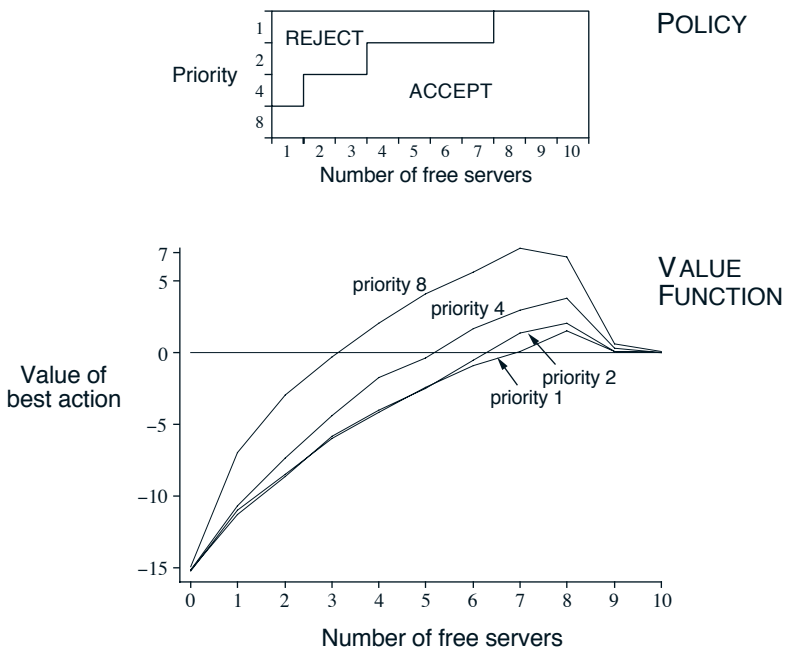


Figure 11.3: The policy and value function found by R-learning on the access-control queuing task after 2 million steps. The drop on the right of the graph is probably due to insufficient data; many of these states were never experienced. The value learned for $\bar{R}$ was about 2.73.

Part III

Frontiers

In this last part of the book we discuss some of the frontiers of reinforcement learning research, including its relationship to neuroscience and animal learning behavior, a sampling of reinforcement learning applications, and prospects for the future of reinforcement learning.

Chapter 12

Psychology

Chapter 13

Neuroscience

Chapter 14

Applications and Case Studies

In this final chapter we present a few case studies of reinforcement learning. Several of these are substantial applications of potential economic significance. One, Samuel's checkers player, is primarily of historical interest. Our presentations are intended to illustrate some of the trade-offs and issues that arise in real applications. For example, we emphasize how domain knowledge is incorporated into the formulation and solution of the problem. We also highlight the representation issues that are so often critical to successful applications. The algorithms used in some of these case studies are substantially more complex than those we have presented in the rest of the book. Applications of reinforcement learning are still far from routine and typically require as much art as science. Making applications easier and more straightforward is one of the goals of current research in reinforcement learning.

14.1 TD-Gammon

One of the most impressive applications of reinforcement learning to date is that by Gerry Tesauro to the game of backgammon (Tesauro, 1992, 1994, 1995). Tesauro's program, *TD-Gammon*, required little backgammon knowledge, yet learned to play extremely well, near the level of the world's strongest grandmasters. The learning algorithm in TD-Gammon was a straightforward combination of the TD(λ) algorithm and nonlinear function approximation using a multilayer neural network trained by backpropagating TD errors.

Backgammon is a major game in the sense that it is played throughout the world, with numerous tournaments and regular world championship matches. It is in part a game of chance, and it is a popular vehicle for waging significant sums of money. There are probably more professional backgammon players

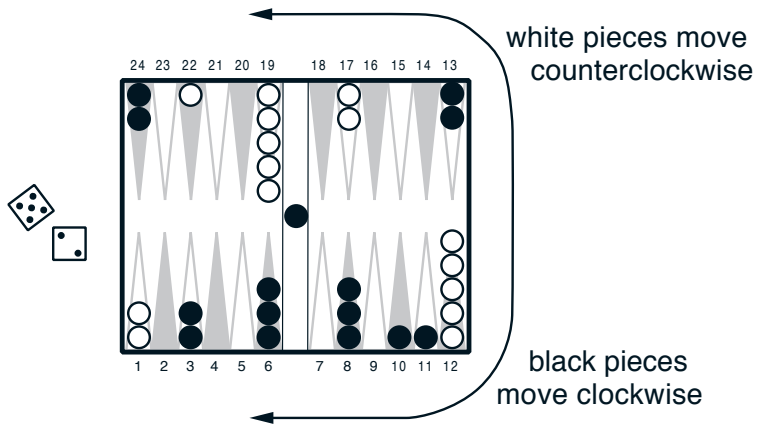


Figure 14.1: A backgammon position

than there are professional chess players. The game is played with 15 white and 15 black pieces on a board of 24 locations, called *points*. Figure 14.1 shows a typical position early in the game, seen from the perspective of the white player.

In this figure, white has just rolled the dice and obtained a 5 and a 2. This means that he can move one of his pieces 5 steps and one (possibly the same piece) 2 steps. For example, he could move two pieces from the 12 point, one to the 17 point, and one to the 14 point. White's objective is to advance all of his pieces into the last quadrant (points 19–24) and then off the board. The first player to remove all his pieces wins. One complication is that the pieces interact as they pass each other going in different directions. For example, if it were black's move in Figure 14.1, he could use the dice roll of 2 to move a piece from the 24 point to the 22 point, "hitting" the white piece there. Pieces that have been hit are placed on the "bar" in the middle of the board (where we already see one previously hit black piece), from whence they reenter the race from the start. However, if there are two pieces on a point, then the opponent cannot move to that point; the pieces are protected from being hit. Thus, white cannot use his 5–2 dice roll to move either of his pieces on the 1 point, because their possible resulting points are occupied by groups of black pieces. Forming contiguous blocks of occupied points to block the opponent is one of the elementary strategies of the game.

Backgammon involves several further complications, but the above description gives the basic idea. With 30 pieces and 24 possible locations (26, counting the bar and off-the-board) it should be clear that the number of possible backgammon positions is enormous, far more than the number of memory elements one could have in any physically realizable computer. The number of moves possible from each position is also large. For a typical dice roll there might be 20 different ways of playing. In considering future moves, such as

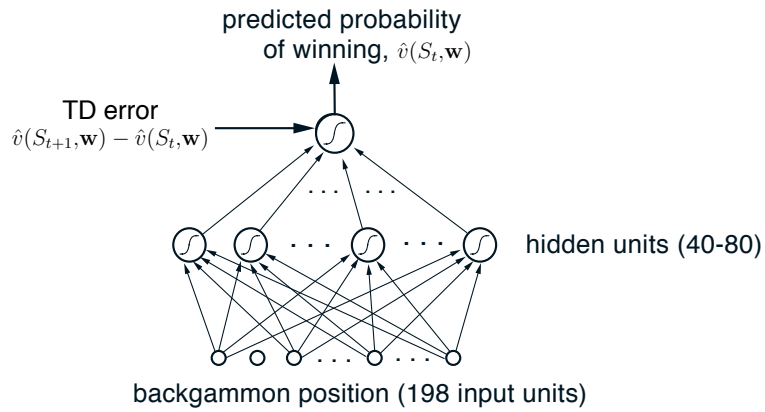


Figure 14.2: The neural network used in TD-Gammon

the response of the opponent, one must consider the possible dice rolls as well. The result is that the game tree has an effective branching factor of about 400. This is far too large to permit effective use of the conventional heuristic search methods that have proved so effective in games like chess and checkers.

On the other hand, the game is a good match to the capabilities of TD learning methods. Although the game is highly stochastic, a complete description of the game's state is available at all times. The game evolves over a sequence of moves and positions until finally ending in a win for one player or the other, ending the game. The outcome can be interpreted as a final reward to be predicted. On the other hand, the theoretical results we have described so far cannot be usefully applied to this task. The number of states is so large that a lookup table cannot be used, and the opponent is a source of uncertainty and time variation.

TD-Gammon used a nonlinear form of TD(λ). The estimated value, $\hat{v}(s)$, of any state (board position) s was meant to estimate the probability of winning starting from state s. To achieve this, rewards were defined as zero for all time steps except those on which the game is won. To implement the value function, TD-Gammon used a standard multilayer neural network, much as shown in Figure 14.2. (The real network had two additional units in its final layer to estimate the probability of each player's winning in a special way called a "gammon" or "backgammon.") The network consisted of a layer of input units, a layer of hidden units, and a final output unit. The input to the network was a representation of a backgammon position, and the output was an estimate of the value of that position.

In the first version of TD-Gammon, TD-Gammon 0.0, backgammon positions were represented to the network in a relatively direct way that involved little backgammon knowledge. It did, however, involve substantial knowledge

of how neural networks work and how information is best presented to them. It is instructive to note the exact representation Tesauro chose. There were a total of 198 input units to the network. For each point on the backgammon board, four units indicated the number of white pieces on the point. If there were no white pieces, then all four units took on the value zero. If there was one piece, then the first unit took on the value 1. If there were two pieces, then both the first and the second unit were 1. If there were three or more pieces on the point, then all of the first three units were 1. If there were more than three pieces, the fourth unit also came on, to a degree indicating the number of additional pieces beyond three. Letting n denote the total number of pieces on the point, if $n > 3$, then the fourth unit took on the value $(n-3)/2$. With four units for white and four for black at each of the 24 points, that made a total of 192 units. Two additional units encoded the number of white and black pieces on the bar (each took the value $n/2$, where n is the number of pieces on the bar), and two more encoded the number of black and white pieces already successfully removed from the board (these took the value $n/15$, where n is the number of pieces already borne off). Finally, two units indicated in a binary fashion whether it was white's or black's turn to move. The general logic behind these choices should be clear. Basically, Tesauro tried to represent the position in a straightforward way, making little attempt to minimize the number of units. He provided one unit for each conceptually distinct possibility that seemed likely to be relevant, and he scaled them to roughly the same range, in this case between 0 and 1.

Given a representation of a backgammon position, the network computed its estimated value in the standard way. Corresponding to each connection from an input unit to a hidden unit was a real-valued weight. Signals from each input unit were multiplied by their corresponding weights and summed at the hidden unit. The output, $h(j)$, of hidden unit j was a nonlinear sigmoid function of the weighted sum:

$$h(j) = \sigma\left(\sum_i w_{ij} x_i\right) = \frac{1}{1 + e^{-\sum_i w_{ij} x_i}},$$

where x_i is the value of the ith input unit and w_{ij} is the weight of its connection to the jth hidden unit. The output of the sigmoid is always between 0 and 1, and has a natural interpretation as a probability based on a summation of evidence. The computation from hidden units to the output unit was entirely analogous. Each connection from a hidden unit to the output unit had a separate weight. The output unit formed the weighted sum and then passed it through the same sigmoid nonlinearity.

TD-Gammon used the gradient-descent form of the TD(λ) algorithm described in Section 9.2, with the gradients computed by the error backpropa-

gation algorithm (Rumelhart, Hinton, and Williams, 1986). Recall that the general update rule for this case is

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha\Big[R_{t+1} + \gamma\hat{v}(S_{t+1},\mathbf{w}_t) - \hat{v}(S_t,\mathbf{w}_t)\Big]\mathbf{e}_t, \tag{14.1}$$

where $\mathbf{w}_t$ is the vector of all modifiable parameters (in this case, the weights of the network) and $\mathbf{e}_t$ is a vector of eligibility traces, one for each component of $\mathbf{w}_t$, updated by

$$\mathbf{e}_t = \gamma\lambda\mathbf{e}_{t-1} + \nabla\hat{v}(S_t,\mathbf{w}_t),$$

with $\mathbf{e}_0 = \mathbf{0}$. The gradient in this equation can be computed efficiently by the backpropagation procedure. For the backgammon application, in which $\gamma = 1$ and the reward is always zero except upon winning, the TD error portion of the learning rule is usually just $\hat{v}(S_{t+1},\mathbf{w}) - \hat{v}(S_t,\mathbf{w})$, as suggested in Figure 14.2.

To apply the learning rule we need a source of backgammon games. Tesauro obtained an unending sequence of games by playing his learning backgammon player against itself. To choose its moves, TD-Gammon considered each of the 20 or so ways it could play its dice roll and the corresponding positions that would result. The resulting positions are *afterstates* as discussed in Section 6.6. The network was consulted to estimate each of their values. The move was then selected that would lead to the position with the highest estimated value. Continuing in this way, with TD-Gammon making the moves for both sides, it was possible to easily generate large numbers of backgammon games. Each game was treated as an episode, with the sequence of positions acting as the states, $S_0, S_1, S_2, \ldots$. Tesauro applied the nonlinear TD rule (14.1) fully incrementally, that is, after each individual move.

The weights of the network were set initially to small random values. The initial evaluations were thus entirely arbitrary. Since the moves were selected on the basis of these evaluations, the initial moves were inevitably poor, and the initial games often lasted hundreds or thousands of moves before one side or the other won, almost by accident. After a few dozen games however, performance improved rapidly.

After playing about 300,000 games against itself, TD-Gammon 0.0 as described above learned to play approximately as well as the best previous backgammon computer programs. This was a striking result because all the previous high-performance computer programs had used extensive backgammon knowledge. For example, the reigning champion program at the time was, arguably, *Neurogammon*, another program written by Tesauro that used a neural network but not TD learning. Neurogammon's network was trained on a large training corpus of exemplary moves provided by backgammon experts, and, in addition, started with a set of features specially crafted for

Program	Hidden Units	Training Games	Opponents	Results
TD-Gam 0.0	40	300,000	other programs	tied for best
TD-Gam 1.0	80	300,000	Robertie, Magriel, ...	−13 pts / 51 games
TD-Gam 2.0	40	800,000	various Grandmasters	−7 pts / 38 games
TD-Gam 2.1	80	1,500,000	Robertie	−1 pt / 40 games
TD-Gam 3.0	80	1,500,000	Kazaros	+6 pts / 20 games

Table 14.1: Summary of TD-Gammon Results

backgammon. Neurogammon was a highly tuned, highly effective backgammon program that decisively won the World Backgammon Olympiad in 1989. TD-Gammon 0.0, on the other hand, was constructed with essentially zero backgammon knowledge. That it was able to do as well as Neurogammon and all other approaches is striking testimony to the potential of self-play learning methods.

The tournament success of TD-Gammon 0.0 with zero backgammon knowledge suggested an obvious modification: add the specialized backgammon features but keep the self-play TD learning method. This produced TD-Gammon 1.0. TD-Gammon 1.0 was clearly substantially better than all previous backgammon programs and found serious competition only among human experts. Later versions of the program, TD-Gammon 2.0 (40 hidden units) and TD-Gammon 2.1 (80 hidden units), were augmented with a selective two-ply search procedure. To select moves, these programs looked ahead not just to the positions that would immediately result, but also to the opponent's possible dice rolls and moves. Assuming the opponent always took the move that appeared immediately best for him, the expected value of each candidate move was computed and the best was selected. To save computer time, the second ply of search was conducted only for candidate moves that were ranked highly after the first ply, about four or five moves on average. Two-ply search affected only the moves selected; the learning process proceeded exactly as before. The most recent version of the program, TD-Gammon 3.0, uses 160 hidden units and a selective three-ply search. TD-Gammon illustrates the combination of learned value functions and decide-time search as in heuristic search methods. In more recent work, Tesauro and Galperin (1997) have begun exploring trajectory sampling methods as an alternative to search.

Tesauro was able to play his programs in a significant number of games against world-class human players. A summary of the results is given in Table 14.1. Based on these results and analyses by backgammon grandmasters (Robertie, 1992; see Tesauro, 1995), TD-Gammon 3.0 appears to be at, or very near, the playing strength of the best human players in the world. It may already be the world champion. These programs have already changed

the way the best human players play the game. For example, TD-Gammon learned to play certain opening positions differently than was the convention among the best human players. Based on TD-Gammon's success and further analysis, the best human players now play these positions as TD-Gammon does (Tesauro, 1995).

14.2 Samuel's Checkers Player

An important precursor to Tesauro's TD-Gammon was the seminal work of Arthur Samuel (1959, 1967) in constructing programs for learning to play checkers. Samuel was one of the first to make effective use of heuristic search methods and of what we would now call temporal-difference learning. His checkers players are instructive case studies in addition to being of historical interest. We emphasize the relationship of Samuel's methods to modern reinforcement learning methods and try to convey some of Samuel's motivation for using them.

Samuel first wrote a checkers-playing program for the IBM 701 in 1952. His first *learning* program was completed in 1955 and was demonstrated on television in 1956. Later versions of the program achieved good, though not expert, playing skill. Samuel was attracted to game-playing as a domain for studying machine learning because games are less complicated than problems "taken from life" while still allowing fruitful study of how heuristic procedures and learning can be used together. He chose to study checkers instead of chess because its relative simplicity made it possible to focus more strongly on learning.

Samuel's programs played by performing a lookahead search from each current position. They used what we now call heuristic search methods to determine how to expand the search tree and when to stop searching. The terminal board positions of each search were evaluated, or "scored," by a value function, or "scoring polynomial," using linear function approximation. In this and other respects Samuel's work seems to have been inspired by the suggestions of Shannon (1950). In particular, Samuel's program was based on Shannon's minimax procedure to find the best move from the current position. Working backward through the search tree from the scored terminal positions, each position was given the score of the position that would result from the best move, assuming that the machine would always try to maximize the score, while the opponent would always try to minimize it. Samuel called this the *backed-up score* of the position. When the minimax procedure reached the search tree's root—the current position—it yielded the best move under the assumption that the opponent would be using the same evaluation criterion,

shifted to its point of view. Some versions of Samuel's programs used sophisticated search control methods analogous to what are known as "alpha-beta" cutoffs (e.g., see Pearl, 1984).

Samuel used two main learning methods, the simplest of which he called *rote learning*. It consisted simply of saving a description of each board position encountered during play together with its backed-up value determined by the minimax procedure. The result was that if a position that had already been encountered were to occur again as a terminal position of a search tree, the depth of the search was effectively amplified since this position's stored value cached the results of one or more searches conducted earlier. One initial problem was that the program was not encouraged to move along the most direct path to a win. Samuel gave it a "a sense of direction" by decreasing a position's value a small amount each time it was backed up a level (called a ply) during the minimax analysis. "If the program is now faced with a choice of board positions whose scores differ only by the ply number, it will automatically make the most advantageous choice, choosing a low-ply alternative if winning and a high-ply alternative if losing" (Samuel, 1959, p. 80). Samuel found this discounting-like technique essential to successful learning. Rote learning produced slow but continuous improvement that was most effective for opening and endgame play. His program became a "better-than-average novice" after learning from many games against itself, a variety of human opponents, and from book games in a supervised learning mode.

Rote learning and other aspects of Samuel's work strongly suggest the essential idea of temporal-difference learning—that the value of a state should equal the value of likely following states. Samuel came closest to this idea in his second learning method, his "learning by generalization" procedure for modifying the parameters of the value function. Samuel's method was the same in concept as that used much later by Tesauro in TD-Gammon. He played his program many games against another version of itself and performed a backup operation after each move. The idea of Samuel's backup is suggested by the diagram in Figure 14.3. Each open circle represents a position where the program moves next, an *on-move* position, and each solid circle represents a position where the opponent moves next. A backup was made to the value of each on-move position after a move by each side, resulting in a second on-move position. The backup was toward the minimax value of a search launched from the second on-move position. Thus, the overall effect was that of a backup consisting of one full move of real events and then a search over possible events, as suggested by Figure 14.3. Samuel's actual algorithm was significantly more complex than this for computational reasons, but this was the basic idea.

Samuel did not include explicit rewards. Instead, he fixed the weight of the most important feature, the *piece advantage* feature, which measured the num-

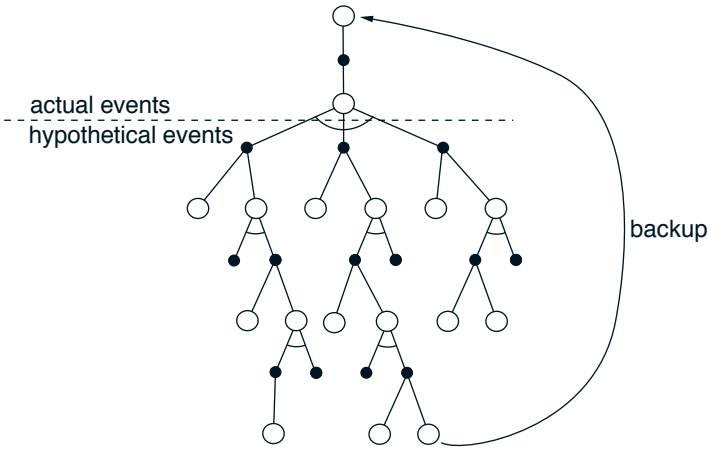


Figure 14.3: The backup diagram for Samuel's checkers player.

ber of pieces the program had relative to how many its opponent had, giving higher weight to kings, and including refinements so that it was better to trade pieces when winning than when losing. Thus, the goal of Samuel's program was to improve its piece advantage, which in checkers is highly correlated with winning.

However, Samuel's learning method may have been missing an essential part of a sound temporal-difference algorithm. Temporal-difference learning can be viewed as a way of making a value function consistent with itself, and this we can clearly see in Samuel's method. But also needed is a way of tying the value function to the true value of the states. We have enforced this via rewards and by discounting or giving a fixed value to the terminal state. But Samuel's method included no rewards and no special treatment of the terminal positions of games. As Samuel himself pointed out, his value function could have become consistent merely by giving a constant value to all positions. He hoped to discourage such solutions by giving his piece-advantage term a large, nonmodifiable weight. But although this may decrease the likelihood of finding useless evaluation functions, it does not prohibit them. For example, a constant function could still be attained by setting the modifiable weights so as to cancel the effect of the nonmodifiable one.

Since Samuel's learning procedure was not constrained to find useful evaluation functions, it should have been possible for it to become worse with experience. In fact, Samuel reported observing this during extensive self-play training sessions. To get the program improving again, Samuel had to intervene and set the weight with the largest absolute value back to zero. His interpretation was that this drastic intervention jarred the program out of local optima, but another possibility is that it jarred the program out of evaluation

functions that were consistent but had little to do with winning or losing the game.

Despite these potential problems, Samuel's checkers player using the generalization learning method approached "better-than-average" play. Fairly good amateur opponents characterized it as "tricky but beatable" (Samuel, 1959). In contrast to the rote-learning version, this version was able to develop a good middle game but remained weak in opening and endgame play. This program also included an ability to search through sets of features to find those that were most useful in forming the value function. A later version (Samuel, 1967) included refinements in its search procedure, such as alpha-beta pruning, extensive use of a supervised learning mode called "book learning," and hierarchical lookup tables called signature tables (Griffith, 1966) to represent the value function instead of linear function approximation. This version learned to play much better than the 1959 program, though still not at a master level. Samuel's checkers-playing program was widely recognized as a significant achievement in artificial intelligence and machine learning.

14.3 The Acrobot

Reinforcement learning has been applied to a wide variety of physical control tasks (e.g., for a collection of robotics applications, see Connell and Mahadevan, 1993). One such task is the *acrobot*, a two-link, underactuated robot roughly analogous to a gymnast swinging on a high bar (Figure 14.4). The first joint (corresponding to the gymnast's hands on the bar) cannot exert torque, but the second joint (corresponding to the gymnast bending at the waist) can. The system has four continuous state variables: two joint positions and two joint velocities. The equations of motion are given in Figure 14.5. This system has been widely studied by control engineers (e.g., Spong, 1994) and machine-learning researchers (e.g., Dejong and Spong, 1994; Boone, 1997).

One objective for controlling the acrobot is to swing the tip (the "feet") above the first joint by an amount equal to one of the links in minimum time. In this task, the torque applied at the second joint is limited to three choices: positive torque of a fixed magnitude, negative torque of the same magnitude, or no torque. A reward of -1 is given on all time steps until the goal is reached, which ends the episode. No discounting is used ($\gamma = 1$). Thus, the optimal value, $v_*(s)$, of any state, s, is the minimum time to reach the goal (an integer number of steps) starting from s.

Sutton (1996) addressed the acrobot swing-up task in an on-line, modelfree context. Although the acrobot was simulated, the simulator was not available

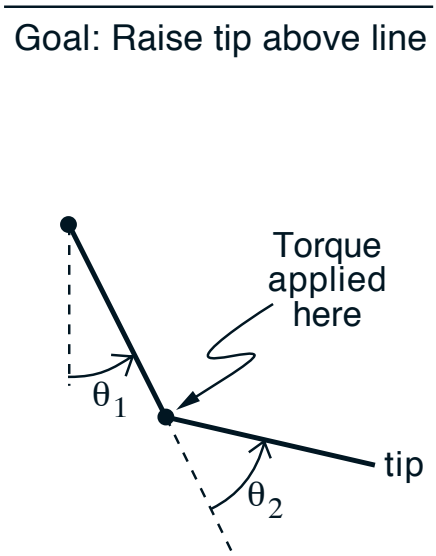


Figure 14.4: The acrobot.

$$
\begin{aligned}
\ddot{\theta}_1 &= -d_1^{-1}(d_2\ddot{\theta}_2 + \phi_1) \\
\ddot{\theta}_2 &= \left(m_2 l_{c2}^2 + I_2 - \frac{d_2^2}{d_1}\right)^{-1}\left(\tau + \frac{d_2}{d_1}\phi_1 - m_2 l_1 l_{c2}\dot{\theta}_1^2 \sin\theta_2 - \phi_2\right) \\
d_1 &= m_1 l_{c1}^2 + m_2(l_1^2 + l_{c2}^2 + 2 l_1 l_{c2}\cos\theta_2) + I_1 + I_2 \\
d_2 &= m_2(l_{c2}^2 + l_1 l_{c2}\cos\theta_2) + I_2 \\
\phi_1 &= -m_2 l_1 l_{c2}\dot{\theta}_2^2 \sin\theta_2 - 2 m_2 l_1 l_{c2}\dot{\theta}_2\dot{\theta}_1 \sin\theta_2 \\
&\quad + \ (m_1 l_{c1} + m_2 l_1) g \cos(\theta_1 - \pi/2) + \phi_2 \\
\phi_2 &= m_2 l_{c2} g \cos(\theta_1 + \theta_2 - \pi/2)
\end{aligned}
$$

Figure 14.5: The equations of motions of the simulated acrobot. A time step of 0.05 seconds was used in the simulation, with actions chosen after every four time steps. The torque applied at the second joint is denoted by $\tau \in \{+1, -1, 0\}$. There were no constraints on the joint positions, but the angular velocities were limited to $\dot{\theta}_1 \in [-4\pi, 4\pi]$ and $\dot{\theta}_2 \in [-9\pi, 9\pi]$. The constants were $m_1 = m_2 = 1$ (masses of the links), $l_1 = l_2 = 1$ (lengths of links), $l_{c1} = l_{c2} = 0.5$ (lengths to center of mass of links), $I_1 = I_2 = 1$ (moments of inertia of links), and $g = 9.8$ (gravity).

for use by the agent/controller in any way. The training and interaction were just as if a real, physical acrobot had been used. Each episode began with both links of the acrobot hanging straight down and at rest. Torques were applied by the reinforcement learning agent until the goal was reached, which always happened eventually. Then the acrobot was restored to its initial rest position and a new episode was begun.

The learning algorithm used was Sarsa(λ) with linear function approximation, tile coding, and replacing traces as in Figure 9.8. With a small, discrete action set, it is natural to use a separate set of tilings for each action. The next choice is of the continuous variables with which to represent the state. A clever designer would probably represent the state in terms of the angular position and velocity of the center of mass and of the second link, which might make the solution simpler and consistent with broad generalization. But since this was just a test problem, a more naive, direct representation was used in terms of the positions and velocities of the links: $\theta_1, \dot{\theta}_1, \theta_2$, and $\dot{\theta}_2$. The two angles are restricted to a limited range by the physics of the acrobot (see Figure 14.5) and the two angles are naturally restricted to $[0, 2\pi]$. Thus, the state space in this task is a bounded rectangular region in four dimensions.

This leaves the question of what tilings to use. There are many possibilities, as discussed in Chapter 9. One is to use a complete grid, slicing the four-dimensional space along all dimensions, and thus into many small four-dimensional tiles. Alternatively, one could slice along only one of the dimensions, making hyperplanar stripes. In this case one has to pick which dimension to slice along. And of course in all cases one has to pick the width of the slices, the number of tilings of each kind, and, if there are multiple tilings, how to offset them. One could also slice along pairs or triplets of dimensions to get other tilings. For example, if one expected the velocities of the two links to interact strongly in their effect on value, then one might make many tilings that sliced along both of these dimensions. If one thought the region around zero velocity was particularly critical, then the slices could be more closely spaced there.

Sutton used tilings that sliced in a variety of simple ways. Each of the four dimensions was divided into six equal intervals. A seventh interval was added to the angular velocities so that tilings could be offset by a random fraction of an interval in all dimensions (see Chapter 9, subsection "Tile Coding"). Of the total of 48 tilings, 12 sliced along all four dimensions as discussed above, dividing the space into $6 \times 7 \times 6 \times 7 = 1764$ tiles each. Another 12 tilings sliced along three dimensions (3 randomly offset tilings each for each of the 4 sets of three dimensions), and another 12 sliced along two dimensions (2 tilings for each of the 6 sets of two dimensions. Finally, a set of 12 tilings depended each on only one dimension (3 tilings for each of the 4 dimensions). This resulted

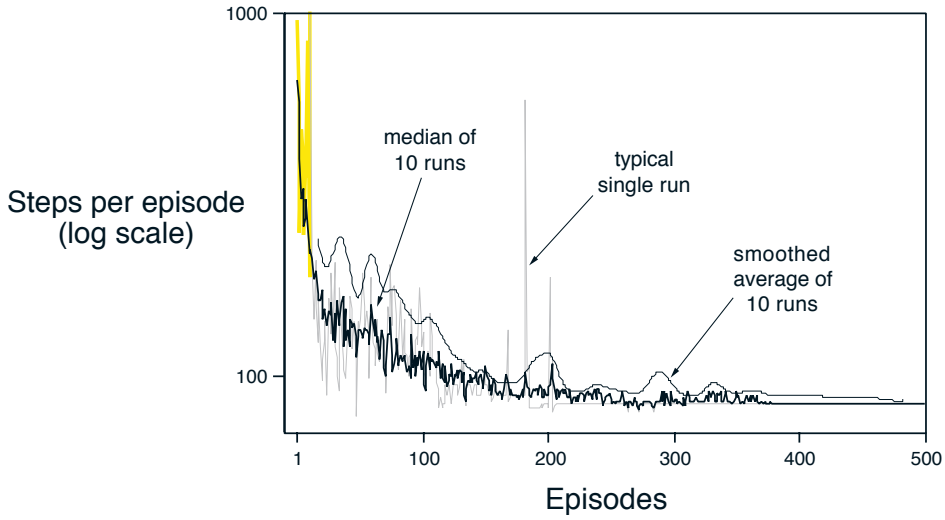


Figure 14.6: Learning curves for Sarsa(λ) on the acrobot task.

in a total of approximately $25,000$ tiles for each action. This number is small enough that hashing was not necessary. All tilings were offset by a random fraction of an interval in all relevant dimensions.

The remaining parameters of the learning algorithm were $\alpha = 0.2/48$, $\lambda = 0.9$, $\epsilon = 0$, and $\mathbf{w}_0 = 0$. The use of a greedy policy ($\varepsilon = 0$) seemed preferable on this task because long sequences of correct actions are needed to do well. One exploratory action could spoil a whole sequence of good actions. Exploration was ensured instead by starting the action values optimistically, at the low value of 0. As discussed in Section 2.7 and Example 9.2, this makes the agent continually disappointed with whatever rewards it initially experiences, driving it to keep trying new things.

Figure 14.6 shows learning curves for the acrobot task and the learning algorithm described above. Note from the single-run curve that single episodes were sometimes extremely long. On these episodes, the acrobot was usually spinning repeatedly at the second joint while the first joint changed only slightly from vertical down. Although this often happened for many time steps, it always eventually ended as the action values were driven lower. All runs ended with an efficient policy for solving the problem, usually lasting about 75 steps. A typical final solution is shown in Figure 14.7. First the acrobot pumps back and forth several times symmetrically, with the second link always down. Then, once enough energy has been added to the system, the second link is swung upright and stabbed to the goal height.

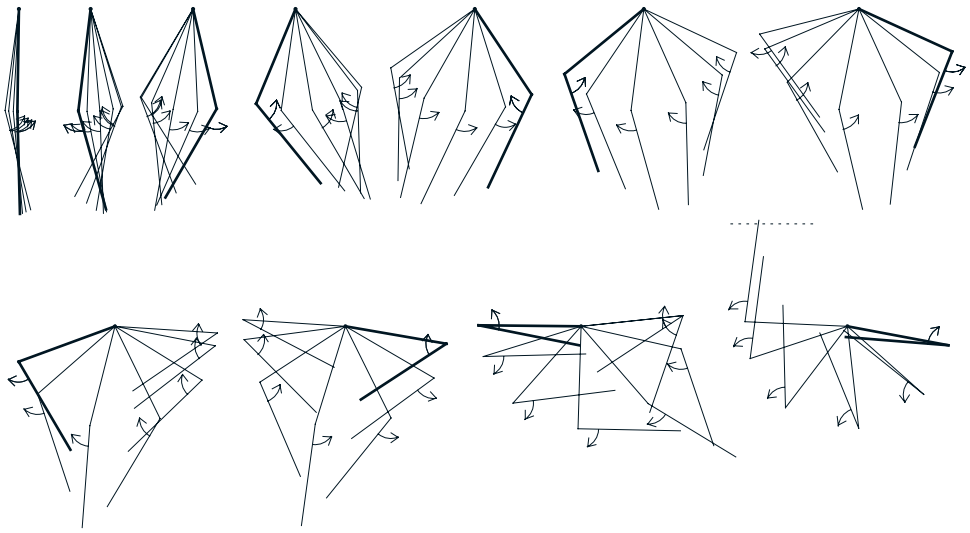


Figure 14.7: A typical learned behavior of the acrobot. Each group is a series of consecutive positions, the thicker line being the first. The arrow indicates the torque applied at the second joint.

14.4 Elevator Dispatching

Waiting for an elevator is a situation with which we are all familiar. We press a button and then wait for an elevator to arrive traveling in the right direction. We may have to wait a long time if there are too many passengers or not enough elevators. Just how long we wait depends on the dispatching strategy the elevators use to decide where to go. For example, if passengers on several floors have requested pickups, which should be served first? If there are no pickup requests, how should the elevators distribute themselves to await the next request? Elevator dispatching is a good example of a stochastic optimal control problem of economic importance that is too large to solve by classical techniques such as dynamic programming.

Crites and Barto (1996; Crites, 1996) studied the application of reinforcement learning techniques to the four-elevator, ten-floor system shown in Figure 14.8. Along the right-hand side are pickup requests and an indication of how long each has been waiting. Each elevator has a position, direction, and speed, plus a set of buttons to indicate where passengers want to get off. Roughly quantizing the continuous variables, Crites and Barto estimated that the system has over 10^{22} states. This large state set rules out classical dynamic programming methods such as value iteration. Even if one state could be backed up every microsecond it would still require over 1000 years to complete just one sweep through the state space.